(1) 개발환경

A. VirtualBox 7.0.20

B. Ubuntu 18.04.2 LTS

C. Linux Kernel Source 4.20.11

(2) CPU 스케줄링 및 정책

운영체제는 여러 개의 프로세스를 효율적으로 실행시키기 위해 CPU를 어떤 순서로 할당할지, 즉 분배할지를 결정해야 한다. 이때 사용하는 정책과 알고리즘이 바로 CPU 스케줄링이다.

A. 필요성

컴퓨터는 여러 프로세스를 동시에 처리하는 멀티태스킹 환경을 지원한다. 하지만 실제로는 CPU가 한 번에 하나의 프로세스만 실행할 수 있다. 따라서 운영체제는 각 프로세스에게 CPU를 번갈아 할당해야만 한다. 이 과정에서 어떤 프로세스에게 어떤 방식으로 우선권을 줄지를 정하는 전략이 필요하다.

B. 핵심 목표

CPU를 최대로 활용, 즉 Idle을 최소화하는 것이다. 이는 메모리와 I/O 병목현을 최소화하는 것과 연결된다. 왜냐하면 Idle을 최소화해도 병목 현상이 심각한 경우 CPU 성능이 저하될 수 있기 때문이다.

C. 스케줄링 Criteria

CPU 스케줄링 Criteria는 다음과 같다. 하지만 아래와 같은 조건을 모두 만족하는 스케줄러를 만드는 것은 현실적으로 불가능하다. 따라서 시스템의 용도와 요구사항을 고려하여 이 중 우선순위를 정해야 한다.

- CPU Utilization: 전체 시스템 시간 중 CPU가 작업을 처리하는 비율을 의미한다.
- 처리량(Throughput): CPU가 단위 시간 당 처리하는 프로세스의 개수이다.
- 응답시간: 프로세스가 입출력을 시작해서 첫 결과가 나오는 데까지 걸리는 시간이다.
- 대기시간: 프로세스가 Ready 큐 내에서 대기하는 시간의 총합이다.
- Turnaround 시간: 프로세스가 시작해서 끝날 때까지 걸리는 시간이다.

D. Preemptive VS Non-preemptive

선점형 스케줄링의 경우 CPU 사용 중에도 더 높은 우선순위의 프로세스가 나타나면 현재 프로세스를 중단시킨다. 응답 시간이 빠르고 사용자 인터페이스에 적합하다는 장점이 있다. 반면 비선점형 스케줄링은 한 번 CPU를 할당 받을 시, 종료하거나 I/O 요청이 있을 때까지 계속 실행한다. 비교적

구현이 간단하고 문맥 전환 횟수가 일반적으로 더 적다는 장점이 있다.

E. 종류

다양한 CPU 스케줄링 정책 중 과제에서 다루는 스케줄링 정책 위주로 한정하여 정리했다.

- FCFS(First Come First Served) 스케줄링 정책
  Ready 큐에 있는 순서대로 CPU를 할당하는 정책이다. 실행시간이 긴 프로세스가 앞에 오면 후속 프로세스가 오랫동안 기다려야 한다는 문제가 있다.
- SRTF(Shortest Remaining Time First) 스케줄링 정책
  남은 실행 시간이 가장 짧은 프로세스를 먼저 처리하는 방식이다. 새 프로세스가 도착하면 현재 실행 중인 프로세스의 남은 시간과 비교한다. 그 다음, 새로 도착한 프로세스의 남은 시간이 더 짧으면 현재 프로세스를 선점하고 새 프로세스를 실행하게 된다.
- Round Robin 스케줄링 정책
  각 프로세스에 동일한 시간 할당량(Time quantum)을 주고 순환하면서 실행한다. 응답시간이 일정해서 인터랙티브 시스템에 적합하다. 다만 시간 할당량이 너무 크면 FCFS처럼 되고, 너무 작으면 문맥전환이 빈번해 오버헤드가 크다.
- Priority with Preemption 스케줄링 정책
  실행 중인 프로세스보다 더 높은 우선순위를 가진 프로세스가 도착했을 때, 현재 실행 중인 프로세스를 중단하고 새로 도착한 프로세스에 CPU를 넘겨주는 방식이다. 중요한 작업을 먼저 처리할 수 있지만, 낮은 우선순위 작업이 계속 기다리게 되어 기아 문제가 발생할 수 있다. 이를 방지하기 위해 Aging 기법(시간이 지남에 따라 우선순위를 높이는 기법)을 사용하기도 한다.

(3) 구현내용

A. ku_cpu.c

새로운 시스템 콜 기능을 수행하는 ku_cpu 함수를 ku_cpu.c 파일에 구현하였다.

- 함께 수정한 파일: **syscall_64.tbl**, **syscalls.h**
  x86_64 아키텍처에서 시스템 콜 번호를 할당하기 위해 arch/x86/entry/syscalls/syscall_64.tbl 파일에 새로운 항목을 추가하였고, 커널에서 시스템 콜 번호를 인식할 수 있도록 include/linux/syscalls.h에 함수 원형을 추가하였다.
- 구현 내용

이 시스템 콜은 단일 진입점에서 다양한 스케줄러를 선택적으로 호출하는 구조를 갖고 있다. **정책 번호(policy = 0:FCFS, 1:SRTF, 2:RR, 3:PP)** 기반 분기 처리를 통해 스케줄링 로직을 모듈화하였다. 내부에서 각 정책에 맞는 handle_request() 함수들이 처리하며, 메인 syscall은 단순 라우팅 역할만 수행한다. 커널 로그 출력으로 결과 확인도 가능하다.

B. FCFS 스케줄링 정책(fcfs.c)

1. CPU 상태 확인
   프로세스 요청이 들어오면, CPU가 비어 있는지 확인한다.
2. FCFS 스케줄링
   CPU가 비어 있다면 바로 실행시킨다. 현재 실행 중인 프로세스와 요청한 PID가 같다면, 작업 시간 0 여부에 따라 종료 및 다음 프로세스를 실행시킨다. 다른 프로세스가 실행 중이라면, 요청을 큐에 추가하고 대기시킨다.
   큐의 경우 큐 동작을 모듈화한 ku_queue.c의 헤더파일 ku_queue.h를 참조하여 연산을 진행한다(push, pop, is_empty 동작).
3. 동시 접근 문제
   동시 접근 문제를 방지하기 위해 spinlock을 사용한다.

- **fcfs_init()**: 실행 중인 프로세스가 없는 상태로 시작한다. 내부 큐, spinlock 초기화를 진행한다.
- **fcfs_handle_request(pid_t pid, const char* name, int job_time)**: FCFS 방식으로 요청을 처리하고 실행 여부를 결정하는 함수이다.
  ① 실행할 프로세스 결정
  ret – 0: 실행 중 1: 대기

```
if (current_pid == IDLE) {
        current_pid = pid;
        ret = 0; // 바로 실행
    } else if (current_pid == pid) {
        if (job_time == 0) {
            if (ku_queue_is_empty()) {
                current_pid = IDLE; // 큐 비었으면 idle
            } else {
                job_t next = ku_queue_pop();
                current_pid = next.pid; // 다음 작업 실행
            }
        }
```

```
        ret = 0; // 현재 프로세스 유지
    } else {
        ku_queue_push(new_job); // 대기 큐에 추가
        ret = 1; // 실행 불가, 대기
    }
```

C. SRTF 스케줄링 정책(srtf.c)

1. 태스크 등록
   프로세스가 들어오면, 이름을 기준으로 기존에 등록된 프로세스인지 확인한다. 없다면 새로 생성해서 리스트에 추가한다 (find_or_create_process() 함수).
2. SRTF 스케줄링
   주어진 남은 시간이 0이면 해당 프로세스를 종료 상태로 표시한다. 종료되지 않은 프로세스들 중에서 가장 남은 시간이 짧은 프로세스를 선택한다. 현재 요청한 프로세스가 그 프로세스라면 CPU 실행을 허용하고, 아니라면 대기시킨다.

- **프로세스 제어 블록 구조체(ku_kcb)**
  프로세스 정보를 담는 핵심 구조체로, 이름, 남은 시간, 상태와 리스트 연결용 멤버 포함한다.
- **select_shortest_remaining():** 종료 상태가 아닌 프로세스 중 가장 짧은 프로세스를 선택한다.
  ① 가장 짧은 프로세스(shortest) 선택
  shortest가 없는 경우는 초기상태를 고려한 조건이다.
  `if (!shortest || curr->remaining_time < shortest->remaining_time)`
- **srts_schedule(name, remain):** 프로세스를 생성하고 실행 가능 여부를 판단하는 SRTF 스케줄러 함수이다.
  ① 프로세스 생성: find_or_create_process()
  ② 프로세스 실행 가능성 판단: select_shortest_remainig()
  Return – 0: 실행 중, 1: 대기, 2: 종료

```
struct ku_pcb *shortest = select_shortest_remaining();
    if (shortest && strcmp(shortest->name, name) == 0) {
        proc->remaining_time--;
        if (proc->remaining_time == 0)
            proc->state = 2; // 종료 상태
        return 0;   //실행 허용 }
    return 1;   //대기 상태 }
```

D. Round Robin 스케줄링 정책(rr.c)

Time quantum은 1초로 설정한다.

1. 태스크 등록
   새로운 프로세스 등록되면, 큐의 끝에 추가된다(add_process() 함수).
2. Round Robin 스케줄링
   큐의 맨 앞에 있는 프로세스인지 확인한다. 맞다면, TIME_QUANTUM 만큼 실행한다. 이후, 해당 프로세스를 큐에서 제거하고, 나머지 프로세스들을 앞으로 시프트한다. 그리고 해당 프로세스가 종료되었으면 큐에서 프로세스를 삭제하고, 아니라면 큐의 맨 뒤에 추가한다 (shift_queue(), add_process() 함수).

- **rr_proc** 구조체: 각 프로세스를 나타내는 구조체이다.
  프로세스명, 남은 실행 시간, state (0: 실행 중, 1: 대기 2: 완료)로 구성된다.
- **rr_schedule(name, job_time)**: 새로운 태스크를 추가하고, 라운드 로빈 스케줄링하는 함수이다.
  ① 큐의 맨 앞 프로세스 확인
  맨 앞 프로세스가 아니면 실행하지 않고, 맞다면 state = 0으로 설정한다.

```
if (strcmp(current->name, name) != 0)
    return 1;
current->state = STATE_RUNNING;
```

  ② Time quantum 종료 여부 확인
  프로세스의 남은 시간이 TIME_QUANTUM만큼 실행하고 난 뒤의 시간과 같은 지 확인한다.

```
int exec_time = (current->remaining_time > TIME_QUANTUM) ?
TIME_QUANTUM : current->remaining_time;
if (current->remaining_time - exec_time == job_time)
```

E. Priority with Preemption 스케줄링 정책(pp.c)

1. 태스크 등록
   새로운 태스크가 등록되면 기존 태스크들과 우선순위를 비교해 선점 여부를 판단합니다.
2. 선점 스케줄링
   현재 실행 중인 태스크보다 우선순위가 높은 태스크가 들어오면 현재 태스크를 대기 상태로 전환시키고, 새 태스크를 실행 상태로 등록합니다.

3. 태스크 완료 및 정리
   실행 중인 태스크가 완료되면, 다음 우선순위가 높은 대기 태스크를 실행합니다. 모든 태스크가 완료되면 내부 리스트와 상태를 초기화 (cleanup_if_all_done() 함수)합니다.

- **select_next_task**: 실행 중인 태스크 종료 시, 대기 중인 태스크 중 가장 우선순위가 높은 태스크 선택하는 함수이다.
  ① 우선순위 높은 태스크(highest) 찾기
  highest가 없는 경우는 초기상태를 고려한 조건이다.
  if (!highest || task_list[i].priority < highest->priority)
- **priority_preemitive_schedule(name, job_time, priority)**: 새로운 태스크를 등록하고, 우선순위에 따라 선점 여부를 결정하는 함수이다.
  ① 동일 프로세스의 job_time 업데이트
  동일 프로세스가 존재하지 않는 경우, 새로운 태스크로 등록한다.

```
if (strcmp(task_list[i].name, name) == 0) {
    task_list[i].job_time = job_time;
    return task_list[i].state;}
```

  ② 선점 조건 확인 및 처리
  state – 0: 실행 중, 1: 대기, 2: 완료

```
if (!currT || new_task.priority < currT->priority) {
    if (currT && currT->state == 0)
        currT->state = 1;   //현재 실행 중인 태스크 대기로 전환
    new_task.state = 0;     //새 태스크 실행으로 등록
    currT = &task_list[task_count];
} else {
    new_task.state = 1;     //대기 상태로 등록}
```

F. test_ku_cpu.c
지시사항에 따라 주어진 소스코드를 완성하고 응답 시간을 출력하도록 수정하였다. policy 변수 설정에 따라 다른 스케줄링 정책을 실행할 수 있다.

G. run
FCFS, SRTF, Round Robin의 실험을 위한 명령어 파일이다.

H. run2
Priority with Preemptive의 실험을 위한 명령어 파일이다.

(4) 실험결과 및 분석

A. 실행결과 – FCFS
A(7s) – B(5s) – C(3s)로 실행된다.

- User

```
hayeon062@hayeon062-VirtualBox:~/os2$ ./run
hayeon062@hayeon062-VirtualBox:~/os2$
Process A : I will use CPU by 7s.

Process B : I will use CPU by 5s.

Process C : I will use CPU by 3s.

Process A : Finish! My response time is 0s and My total wait time is 0s.
Process B : Finish! My response time is 6s and My total wait time is 6s.
Process C : Finish! My response time is 10s and My total wait time is 10s.
```

- Kernel (일부)

```
[   61.059712] Working: A
[   61.161013] Working: A
[   61.266206] Working: A
[   61.367568] Working: A
[   61.468761] Working: A
[   61.569287] Working: A
[   61.672925] Working: A
[   61.773960] Working: A
[   61.918115] Working: A
[   62.019069] Working: A
[   62.064910] Working Denied: B
[   62.124292] Working: A
[   62.166979] Working Denied: B
[   62.225775] Working: A
[   62.268104] Working Denied: B
[   62.332203] Working: A
[   62.369066] Working Denied: B
[   62.433260] Working: A
[   62.469945] Working Denied: B
[   62.534812] Working: A
[   62.570621] Working Denied: B
[   62.635848] Working: A
[   62.672289] Working Denied: B
[   62.737414] Working: A
[   62.773978] Working Denied: B
[   62.840292] Working: A
[   62.875393] Working Denied: B
```

```
[   65.824344] Working: A
[   65.874605] Working Denied: B
[   65.924305] Working Denied: C
[   65.926755] Working: A
[   65.981225] Working Denied: B
[   66.027317] Working Denied: C
[   66.031857] Working: A
[   66.090096] Working Denied: B
[   66.128377] Working Denied: C
[   66.132568] Working: A
[   66.196579] Working Denied: B
[   66.231705] Working Denied: C
[   66.241486] Working: A
[   66.300099] Working Denied: B
[   66.336399] Working Denied: C
[   66.347825] Working: A
[   66.403358] Working Denied: B
[   66.437978] Working Denied: C
[   66.448183] Working: A
[   66.510923] Working Denied: B
[   66.539028] Working Denied: C
[   66.548919] Working: A
[   66.615358] Working Denied: B
[   66.640193] Working Denied: C
[   66.649735] Working: A
[   66.734336] Working Denied: B
[   66.741546] Working Denied: C
```

B. 실행결과 – SRTF

A(1s) – B(1s) – C(3s) – B(4s) – A(6s)로 실행된다.

- User

```
hayeon062@hayeon062-VirtualBox:~/os2$ ./run
hayeon062@hayeon062-VirtualBox:~/os2$
Process A : I will use CPU by 7s.

Process B : I will use CPU by 5s.

Process C : I will use CPU by 3s.

Process C : Finish! My response time is 0s and My total wait time is 0s.
Process B : Finish! My response time is 0s and My total wait time is 3s.
Process A : Finish! My response time is 0s and My total wait time is 8s.
```

- Kernel (일부)

```
[   49.390295] Working: A
[   49.490902] Working: A
[   49.592048] Working: A
[   49.692582] Working: A
[   49.794527] Working: A
[   49.895570] Working: A
[   49.996786] Working: A
[   50.097357] Working: A
[   50.198994] Working: A
[   50.300561] Working: A
[   50.390743] Working: B
[   50.401239] Working Denied: A
[   50.491313] Working: B
[   50.501771] Working Denied: A
[   50.592531] Working: B
[   50.602160] Working Denied: A
[   50.693003] Working: B
[   50.702853] Working Denied: A
[   50.793610] Working: B
[   50.803856] Working Denied: A
[   50.894847] Working: B
[   50.904335] Working Denied: A
[   50.995487] Working: B
[   51.004767] Working Denied: A
[   51.096700] Working: B
[   51.105871] Working Denied: A
[   51.199468] Working: B
[   51.206701] Working Denied: A
[   51.300505] Working: B
[   51.307608] Working Denied: A
```

```
[   52.010731] Working: C
[   52.016087] Working Denied: A
[   52.109535] Working Denied: B
[   52.112333] Working: C
[   52.139212] Working Denied: A
[   52.211082] Working Denied: B
[   52.214159] Working: C
[   52.240686] Working Denied: A
[   52.312816] Working Denied: B
[   52.315167] Working: C
[   52.344472] Working Denied: A
[   52.414124] Working Denied: B
[   52.416130] Working: C
[   52.446366] Working Denied: A
[   52.514626] Working Denied: B
[   52.516839] Working: C
[   52.547668] Working Denied: A
[   52.615989] Working Denied: B
[   52.617859] Working: C
[   52.651345] Working Denied: A
[   52.717164] Working Denied: B
[   52.719361] Working: C
[   52.752714] Working Denied: A
[   52.818223] Working Denied: B
[   52.821206] Working: C
[   52.854292] Working Denied: A
[   52.919712] Working Denied: B
[   52.922645] Working: C
[   52.956221] Working Denied: A
[   53.022209] Working Denied: B
```

C. 실행결과 – Round Robin

A(1s) – B(1s) – A(1s) – C(1s) – B(1s)…. – A(1s)로 실행된다.

- User

```
hayeon062@hayeon062-VirtualBox:~/os2$ ./run
hayeon062@hayeon062-VirtualBox:~/os2$
Process A : I will use CPU by 7s.

Process B : I will use CPU by 5s.

Process C : I will use CPU by 3s.

Process C : Finish! My response time is 1s and My total wait time is 6s.
Process B : Finish! My response time is 0s and My total wait time is 8s.
Process A : Finish! My response time is 0s and My total wait time is 9s.
```

- Kernel (일부)

```
[   43.170631] Turn Over- - - -> A
[   43.236507] Working: C
[   43.337107] Working: C
[   43.511778] Working: C
[   43.612234] Working: C
[   43.712819] Working: C
[   43.813340] Working: C
[   43.919269] Working: C
[   44.020109] Working: C
[   44.130555] Working: C
[   44.231009] Working: C
[   44.354387] Turn Over- - - -> C
[   44.455915] Working: B
[   44.558268] Working: B
[   44.658879] Working: B
[   44.760167] Working: B
[   44.862584] Working: B
[   44.964030] Working: B
[   45.065220] Working: B
[   45.168203] Working: B
[   45.269682] Working: B
[   45.371049] Working: B
[   45.471857] Turn Over- - - -> B
[   45.477665] Working: A
[   45.578511] Working: A
[   45.680413] Working: A
[   45.783179] Working: A
[   45.884972] Working: A
[   45.990696] Working: A
[   46.094267] Working: A
[   46.211112] Working: A
[   46.312425] Working: A
[   46.416747] Working: A
```

```
[   94.965148] Process Finished: C
[   94.965185] Working: B
[   95.073117] Working: B
[   95.209016] Working: B
[   95.311861] Working: B
[   95.413653] Working: B
[   95.515775] Working: B
[   95.627394] Working: B
[   95.728100] Working: B
[   95.840273] Working: B
[   95.942282] Working: B
[   96.043766] Turn Over----> B
[   96.043800] Working: A
[   96.146726] Working: A
[   96.251456] Working: A
[   96.352673] Working: A
[   96.456184] Working: A
[   96.556916] Working: A
[   96.661485] Working: A
[   96.762368] Working: A
[   96.863404] Working: A
[   96.964848] Working: A
[   97.066831] Turn Over----> A
[   97.066870] Working: B
[   97.172941] Working: B
[   97.282400] Working: B
[   97.383816] Working: B
[   97.487648] Working: B
[   97.590192] Working: B
[   97.695692] Working: B
[   97.798380] Working: B
[   97.906317] Working: B
[   98.011671] Working: B
[   98.113709] Process Finished: B
```

D. 실행결과 – Priority with Preemption

A(1s) – B(5s) – A(6s) – C(3s)로 실행된다.

- User

```
hayeon062@hayeon062-VirtualBox:~/os2$ ./run2
hayeon062@hayeon062-VirtualBox:~/os2$
Process A : I will use CPU by 7s.

Process B : I will use CPU by 5s.

Process C : I will use CPU by 3s.

Process B : Finish! My response time is 0s and My total wait time is 0s.
Process A : Finish! My response time is 0s and My total wait time is 5s.
Process C : Finish! My response time is 10s and My total wait time is 10s.
```

- Kernel

```
[  118.475509] Working: A
[  118.576221] Working: A
[  118.677946] Working: A
[  118.780340] Working: A
[  118.881968] Working: A
[  118.982917] Working: A
[  119.084397] Working: A
[  119.188360] Working: A
[  119.289737] Working: A
[  119.376168] Working: B
[  119.390276] Working Denied: A
[  119.480665] Working: B
[  119.491644] Working Denied: A
[  119.584237] Working: B
[  119.593076] Working Denied: A
[  119.685463] Working: B
[  119.695369] Working Denied: A
```

```
[  124.066742] Working: B
[  124.088956] Working Denied: A
[  124.127622] Working Denied: C
[  124.169114] Working: B
[  124.189003] Working Denied: A
[  124.228669] Working Denied: C
[  124.270274] Working: B
[  124.289060] Working Denied: A
[  124.328876] Working Denied: C
[  124.370861] Working: B
[  124.389618] Working Denied: A
[  124.429518] Working Denied: C
[  124.474075] Working: B
[  124.494195] Working: A
[  124.534315] Working Denied: C
[  124.594828] Working: A
[  124.634551] Working Denied: C
[  124.697212] Working: A
[  124.742545] Working Denied: C
```

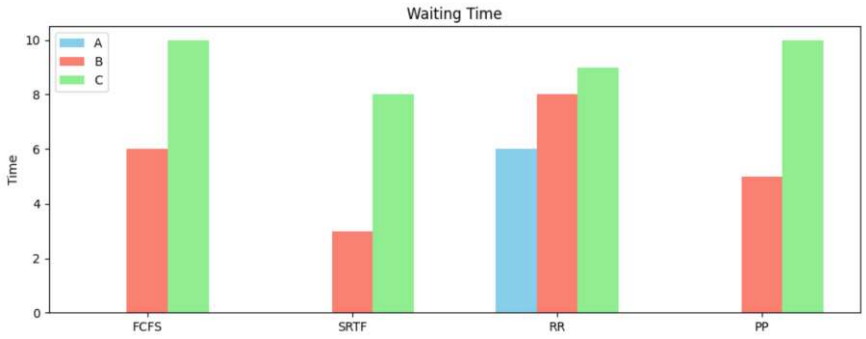
```
[  131.069110] Working: C
[  131.170605] Working: C
[  131.275649] Working: C
[  131.376276] Working: C
[  131.483945] Working: C
[  131.592678] Working: C
[  131.694876] Working: C
[  131.803185] Working: C
[  131.903702] Working: C
[  132.004760] Working: C
[  132.113018] Working: C
[  132.213892] Working: C
[  132.315444] Working: C
[  132.415548] Working: C
[  132.525131] Working: C
[  132.626437] Working: C
[  132.727453] Working: C
[  132.911085] Working: C
```

E. 분석

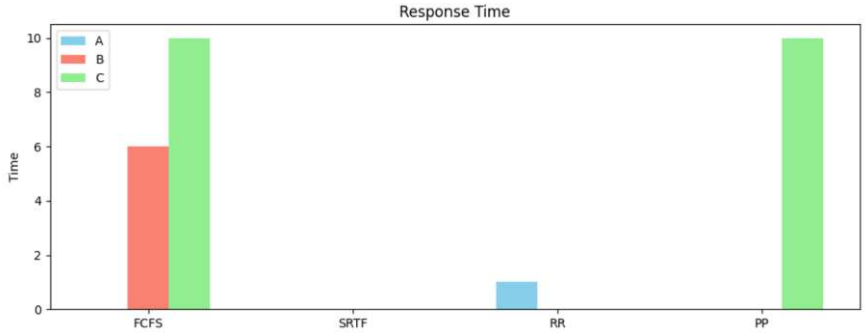
- 각 스케줄링 정책 하의 프로세스별 시간

순서대로 FCFS, SRTF, Round Robin, Priority with Preemption 정책에 관한 프로세스별 시간이며, 프로세스는 프로세스 A, B, C로 구성된다.

1. 대기시간



2. 응답시간



- 스케줄러 정책 별 장단점/특성/적절한 적용 상황 논의
  1. FCFS 스케줄링 정책
     구현이 간단하지만, 데이터를 통해 알 수 있듯이 대기시간이 길어질 수 있다. 특히 긴 작업(A)이 먼저 오면 뒤에 프로세스(B,C)의 대기시간이 길어진다. 따라서 실행시간 차이가 적거나 순서를 엄격히 지켜야 하는 상황에 적절하다.
  2. SRTF 스케줄링 정책
     데이터를 통해 알 수 있듯이 대기/응답시간이 상대적으로 낮고 균등하다. 즉, 평균 대기/응답시간을 최소화한다는 장점이 있다. 하지만 기아 발생 위험과 복잡한 구현이 단점이다. 빠른 반응 시간이 중요한 실시간 처리 환경에 적합하다.
  3. Round Robin 스케줄링 정책
     데이터를 통해 응답/대기시간이 중간 정도이며 프로세스들이 공평하게 분배 받아 어느정도 균일한 응답을 제공함을 알 수 있다. 즉, 장점은 공정성 보장이지만 시간 할당량 설정에 따라 성능 차이가 난다는 단점이 있다. 따라서 다중 사용자 시스템이나 인터랙티브 시스템에 적합하다.
  4. Priority with Preemption 스케줄링 정책

데이터를 통해 응답시간은 높은 우선순위 프로세스가 매우 짧고, 낮은 우선순위 프로세스는 긴 대기시간이 발생함을 알 수 있다. 따라서 중요한 작업을 빠르게 처리하지만, 낮은 우선순위 프로세스는 기아 상태에 빠질 수 있다. 우선순위가 중요한 긴급/실시간 처리 시스템의 경우에 적절할 것 같다.

(5) 과제 수행 중 발생한 문제점 및 해결방법

별다른 문제점 없이 수행 완료하였다.